

Ros2 Day1 hw3 AI 사용 보고서

휴머노이드 수습단원 이주원

목차

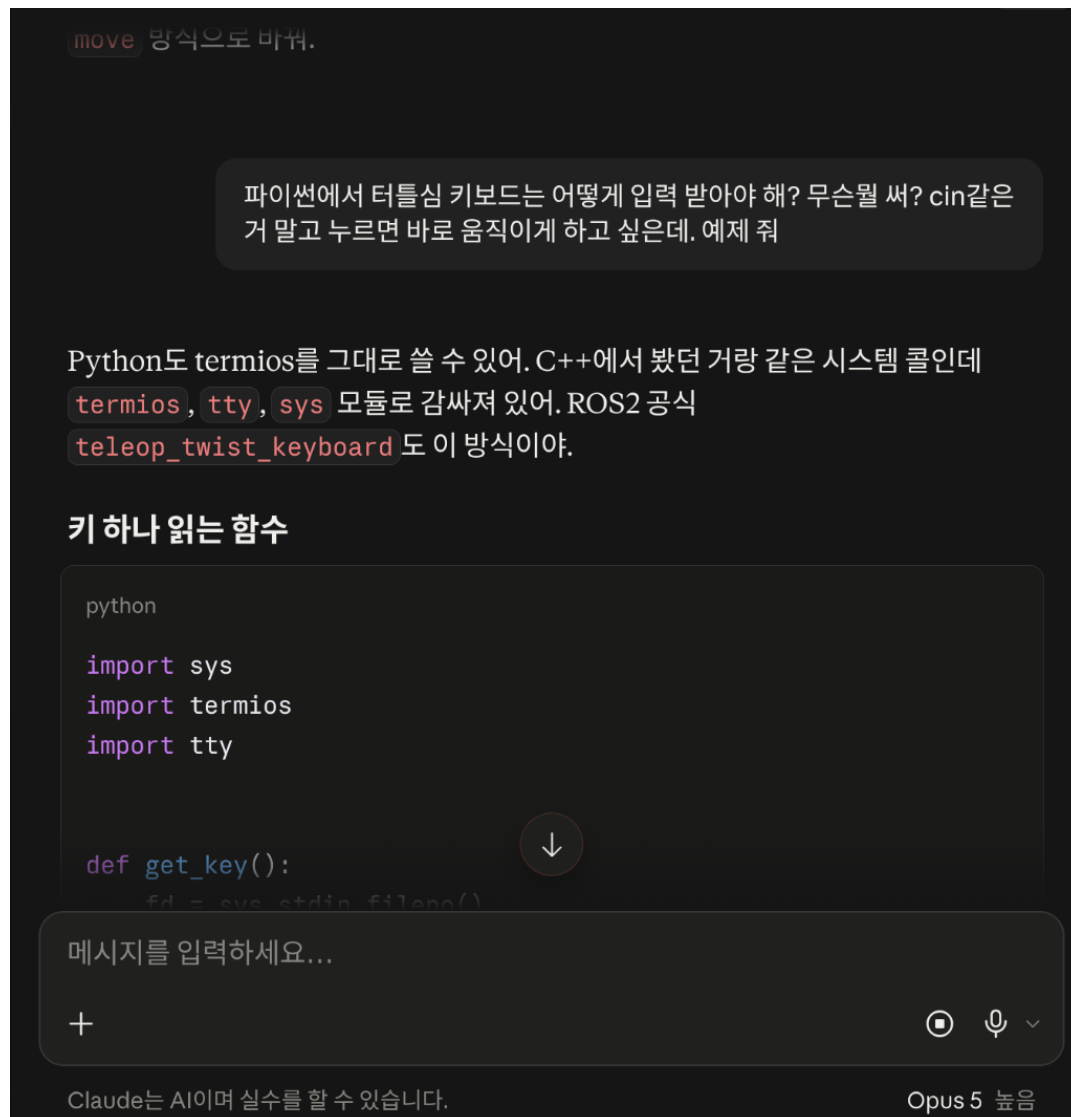
1. ai사용 부분
2. 대화내용
3. 깨달은 점

1. ai 사용 부분

이 과제에서 ai가 사용된 부분은 첫 번째로 키보드 입력 처리 방식이다. 과제 요구사항이 W, A, S, D 키를 누르면 거북이가 해당 도형을 그리는 것이었는데, 파이썬에서 기본으로 제공하는 입력 함수는 엔터를 눌러야 값이 넘어오는 방식이라 요구사항과 맞지 않았다. 키를 누르는 즉시 반응하게 만들려면 어떤 방법을 써야 하는지, 그리고 파이썬에도 그런 기능이 있는지 자체를 몰라서 사용 가능한 방법과 예제를 요청하였다. 터미널이 입력을 한 줄씩 모아두었다가 넘겨준다는 사실 자체를 이 과정에서 처음 인지하게 되었기 때문에, 검색 키워드를 잡기 어려워 ai에게 물어보는 쪽이 빨랐다.

두 번째는 도형별로 선 색을 다르게 하는 방법이다. 도형 네 개가 같은 색으로 겹쳐 그려지니 결과 화면에서 구분이 잘 되지 않아, 각 도형마다 펜 색과 굵기를 바꿔주고 싶었다. 그런데 지금까지 배운 것은 토픽을 발행해서 속도를 보내는 방식뿐이었고, 펜 설정은 토픽 목록에 보이지 않았다. 이것이 다른 종류의 통신으로 처리되는 것 같다는 짐작은 했지만 실제로 어떤 형태로 호출해야 하는지 몰라서, 사용 절차와 예제를 알려달라고 요청하였다.

2. 대화 내용



파이썬에서도 터미널 설정을 직접 다루는 표준 모듈을 쓸 수 있고, 이것이 C++에서 쓰던 것과 같은 기능이라는 답을 받았다. 방법은 현재 터미널 설정을 먼저 백업해두고, 입력을 모아두지 않는 모드로 잠시 바꾼 뒤 글자 하나를 읽고, 끝나면 반드시 원래 설정으로 되돌리는 순서다. 되돌리는 부분은 중간에 오류가 나더라도 반드시 실행되도록 작성해야 한다는 설명도 함께 받았다. ROS2에서 기본으로 제공하는 키보드 조작 예제도 같은 방식으로 만들어져 있다고 한다.

setpen 어떻게 쓰는지 예제 알려줘

토픽이랑 패턴이 비슷한데 세 단계야: 클라이언트 만들기 → 요청 객체 채우기 → 보내기.

1. import 추가

```
python  
  
from turtlesim.srv import SetPen
```

`msg`가 아니라 `**srv**`야. 서비스니까.

2. 생성자에 클라이언트 추가

```
python  
  
def __init__(self):  
    super().__init__('teleop_publisher')
```

메시지를 입력하세요...

+



Claude는 AI이며 실수를 할 수 있습니다.

Opus 5 높음

펜 설정은 토픽이 아니라 서비스로 제공되며, 사용 절차는 크게 세 단계라는 답을 받았다. 먼저 해당 서비스에 대한 클라이언트를 생성자에서 만들어두고, 사용할 때 요청 객체를 만들어 색과 굵기 값을 채운 다음, 그 요청을 보내면 된다. 불러올 때도 메시지가 들어 있는 경로가 아니라 서비스가 들어 있는 경로에서 가져와야 한다는 점을 짚어주었다. 전체적인 흐름은 토픽을 쓸 때와 비슷하지만, 값을 계속 흘려보내는 것이 아니라 한 번 요청하고 끝난다는 점이 다르다는 설명이었다.

3. 깨달은 점

-터미널이 입력을 곧바로 프로그램에 넘겨주는 것이 아니라, 한 줄이 완성될 때까지 모아두었다가 전달한다는 것을 알게 되었다. 평소에 엔터를 누르는 것이 당연하다고만 여겼는데 그것이 사실은 터미널의 동작 방식 때문이었고, 이 설정을 잠시 바꿔주면 키를 누르는 즉시 프로그램이 반응할 수 있다는 점이 새로웠다. 또한 설정을 바꿔 쓴 뒤에는 반드시 원래대로 되돌려야 하며, 그렇지 않으면 프로그램이 끝난 뒤에도 터미널이 이상하게 동작한다는 것도 함께 알게 되었다.

-ROS2에서 노드끼리 주고받는 방법이 토픽 하나만 있는 것이 아니라는 사실을 깨달았다. 속도처럼 계속 흘러보내는 값은 토픽이 어울리지만, 펜 색을 바꾸는 것처럼 한 번 요청하고 처리되면 끝나는 동작은 서비스라는 다른 방식으로 되어 있었다. 원하는 기능이 토픽 목록에 보이지 않는다고 해서 불가능한 것이 아니라, 어떤 종류의 통신으로 제공되는지를 먼저 확인해야 한다는 점을 배웠다.

-토픽과 서비스가 쓰는 방법이 아예 다른 것은 아니고, 무엇을 만들고 값을 채워서 보낸다는 큰 틀은 비슷하다는 것을 확인하였다. 다만 불러오는 경로가 메시지 쪽이 아니라 서비스 쪽이라는 차이가 있어서, 이 부분을 헛갈리면 애초에 코드가 실행되지 않는다는 점을 알게 되었다.

-요청을 보낸 직후에 바로 도형을 그리기 시작하면 펜 설정이 아직 반영되지 않은 채로 선이 그려질 수 있다는 것도 실습 중에 확인하였다. 명령을 보내는 것과 그 명령이 실제로 적용되는 것 사이에 시간 차가 있다는 점을 감안해서 코드를 짜야 한다는 것을 깨달았다